

Q1. Уникальные рейсы и сортировка

1. `SELECT DISTINCT flight\_no ...` vs `SELECT flight\_no ... GROUP BY flight\_no ...`

возвращают уникальные значения `flight\_no`, а `ORDER BY flight\_no` задаёт одинаковый порядок. `DISTINCT` и `GROUP BY` в PostgreSQL одинаково видят `NULL` как отдельную группу.

. PostgreSQL может выполнить:

- `DISTINCT` через `HashAggregate`, либо через `Sort` + `Unique`
- `GROUP BY` через `HashAggregate` или `GroupAggregate`.

2. Почему достаточно `ORDER BY flight\_no` без `ASC`?

Потому что в SQL по умолчанию сортировка идёт по возрастанию: `ORDER BY flight\_no` означает `ORDER BY flight\_no ASC`. Для `ASC` в PostgreSQL также по умолчанию `NULLS LAST`.

Сортировка строк с разным регистром зависит от правил сравнения строк **collation**. Например:

сортировка идёт по байтам: `PG0001` будет раньше `pg0001`, потому что заглавные буквы имеют меньший код

Q2. Аэропорты и выручка за билеты

1. Один пассажир купил два билета на один рейс

Если пассажир купил два разных билета на один и тот же рейс, то в `ticket\_flights` будет две строки.

SUM корректно просуммирует стоимость обоих билетов

COUNT посчитает две строки, то есть один и тот же пассажир будет учтён дважды.

Если нужно считать именно уникальных пассажиров

COUNT(DISTINCT t.passenger_id)

2. Рейс есть в flights, но билетов нет

Если использовать INNER JOIN между flights и ticket_flights, такой рейс просто исчезнет из результата, потому что для него нет совпадающих строк.

Чтобы рейс попал в выборку с нулевой выручкой, нужно использовать LEFT JOIN:

SELECT

f.flight_id,

COALESCE(SUM(tf.amount), 0) AS revenue,

COUNT(DISTINCT t.passenger_id) AS passengers

FROM flights f

LEFT JOIN ticket_flights tf ON tf.flight_id = f.flight_id

LEFT JOIN tickets t ON t.ticket_no = tf.ticket_no

GROUP BY f.flight_id;

LEFT JOIN сохраняет все рейсы;

COALESCE(SUM(tf.amount), 0) превращает NULL в 0;

COUNT(DISTINCT t.passenger_id) не считает NULL-пассажиров, поэтому для рейса без билетов вернёт 0.

INNER JOIN здесь не подходит, потому что он отбрасывает рейсы без совпадений.

Q3. Отменённые рейсы и города

1. Аэропорт, который вообще не фигурирует в flights

Город, в котором есть аэропорт, но из него не было ни одного рейса должен попасть в результат. не было ни одного отменённого рейса. Такой город нельзя терять.

Надёжный вариант идти от airports и использовать NOT EXISTS:

SELECT DISTINCT a.city

FROM airports a

WHERE NOT EXISTS (

SELECT 1

FROM flights f

WHERE f.departure_airport = a.airport_code

AND f.status = 'Cancelled'

);

включает аэропорты, которых нет в flights;

исключает города, из которых был хотя бы один отменённый рейс.

NOT IN тоже мог бы включить аэропорты без рейсов,

2. Ловушка NOT IN с NULL

Если написать:

city NOT IN (

SELECT departure_airport

FROM flights

WHERE status = 'Cancelled'

)

и подзапрос вернёт хотя бы один NULL, то условие NOT IN перестанет работать предсказуемо. Формально x NOT IN (a, b, NULL)

Q4. Места на борту и дубликаты

1. Один пассажир получил два посадочных талона

Если один и тот же passenger_id ошибочно получил два посадочных талона на разные места, то при соединении boardings и tickets он может появиться несколько раз.

Чтобы вывести уникальных пассажиров на рейсе, можно использовать:

```
SELECT DISTINCT
```

```
    b.flight_id,
```

```
    t.passenger_id
```

```
FROM boardings b
```

```
JOIN tickets t ON t.ticket_no = b.ticket_no;
```

или:

```
SELECT
```

```
    b.flight_id,
```

```
    t.passenger_id
```

```
FROM boardings b
```

```
JOIN tickets t ON t.ticket_no = b.ticket_no
```

```
GROUP BY b.flight_id, t.passenger_id;
```

Если нужна только дедупликация DISTINCT читается проще. Если дополнительно нужны агрегаты GROUP BY.

2. Пассажир сдал билет, но есть посадочный талон

Ситуация аномальная: пассажира уже нет в ticket_flights, но он есть в boardings.

Если запрос соединяет только tickets и boardings по ticket_no, то пассажир может попасть в список, если строка в tickets ещё существует. Если же запрос обязательно идёт через ticket_flights, то он не попадёт в список, потому что там нет его билета на рейс.

То есть результат зависит от выбранного пути соединения:

tickets JOIN boardings пассажир может остаться в списке;

ticket_flights JOIN boardings пассажир исчезнет.

Если нужно выявить такие аномалии, можно использовать LEFT JOIN к ticket_flights и фильтр tf.ticket_no IS NULL. Если же список должен строиться только по фактически проданным и действительным билетам, нужно требовать наличие строки в ticket_flights.

Q5. DDL и блокировки на живой таблице

1. ALTER TABLE flights ADD COLUMN weather_data JSONB; и долгий SELECT

ALTER TABLE ADD COLUMN пытается взять блокировку ACCESS EXCLUSIVE на таблицу flights. Эта блокировка конфликтует с ACCESS SHARE, которую держит обычный SELECT.

Если другой пользователь уже выполнил:

```
BEGIN;
```

```
SELECT COUNT(*) FROM flights;
```

он держит ACCESS SHARE до конца транзакции. Поэтому DDL-команда будет ждать завершения этой транзакции.

При этом после того, как ALTER TABLE встанет в очередь на ACCESS EXCLUSIVE, новые SELECT тоже могут начать ждать, потому что PostgreSQL старается не допускать starvation: запросы на разделяемые блокировки могут пропускать вперёд уже ожидающую эксклюзивную блокировку.

2. ADD COLUMN is_delayed BOOLEAN DEFAULT false в PostgreSQL 11+

В PostgreSQL 11+ добавление колонки с константным DEFAULT не требует перезаписи всей таблицы. Значение по умолчанию хранится в системном каталоге:

в pg_attribute выставляется atthasmissing = true;

в attmissingval сохраняется значение по умолчанию.

Для старых строк значение не записывается в каждую heap-строку. При чтении PostgreSQL понимает, что колонка «отсутствует» в старой строке, и подставляет значение из attmissingval. Новые строки уже получают обычное значение.

Если потом изменить DEFAULT на true:

```
ALTER TABLE flights ALTER COLUMN is_delayed SET DEFAULT true;
```

то:

старые строки, добавленные до появления колонки, всё ещё будут возвращать false через механизм missing value;

новые INSERT после изменения начнут получать true;

параллельные INSERT, которые выполнялись до commit изменения DEFAULT, увидят старое значение, а после commit новое.

Чтобы изменить уже существующие строки, нужен отдельный UPDATE.

3. CREATE INDEX idx_flights_status ON flights(status);

Обычный CREATE INDEX берёт на таблицу блокировку SHARE. Она совместима с чтением (ACCESS SHARE), но конфликтует с ROW EXCLUSIVE, которую берут INSERT, UPDATE, DELETE. Поэтому на время построения индекса запись в таблицу блокируется.

Чтобы не блокировать запись, используют:

```
CREATE INDEX CONCURRENTLY idx_flights_status ON flights(status);
```

построение идёт дольше;

выполняется несколько проходов по таблице;

сильнее нагружает I/O;

может завершиться ошибкой и оставить INVALID индекс;

нельзя выполнять внутри транзакции.

Q6. Агрегация и NULL в вычислениях

1. AVG(actual_departure - scheduled_departure)

Если actual_departure IS NULL, например рейс отменён, то выражение:

sql

```
actual_departure - scheduled_departure
```

даёт NULL. Агрегатная функция AVG игнорирует NULL-значения. Значит, средняя задержка будет посчитана только по рейсам, у которых есть фактическое время вылета.

Это может исказить результат: отменённые рейсы просто выпадут из расчёта. Если нужно считать их как нулевую задержку, можно использовать:

```
AVG(COALESCE(actual_departure - scheduled_departure, INTERVAL '0'))
```

Если все значения NULL, AVG вернёт NULL.

2. COUNT(aircraft_code) vs COUNT(*)

COUNT(*) считает все строки.

COUNT(aircraft_code) считает только строки, где aircraft_code IS NOT NULL.

Если на колонке aircraft_code стоит ограничение NOT NULL, результаты совпадут. Если ограничения нет и есть NULL-значения, то:

```
COUNT(aircraft_code) < COUNT(*)
```

Например, если в таблице 100 рейсов и у 5 не указан aircraft_code, то COUNT(*) = 100, а COUNT(aircraft_code) = 95.